

Bubble Sort

Learn how repeated neighbour swaps put a row of values in order.

How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.

Start with the idea

Bubble sort walks along a row and checks neighbouring values. If the larger neighbour comes first, swap them: each takes the other's position. One full walk is called a pass.

A small example

Make the first pass through [4, 2, 5, 1, 3].

Compare 4 and 2. Swap them because 4 is larger: [2, 4, 5, 1, 3].

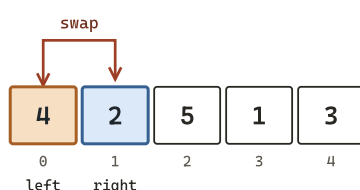
In the next illustration, read each neighbour comparison in order, then look at the row after the pass. The completed positions on the right can be left alone.

The whole process at a glance

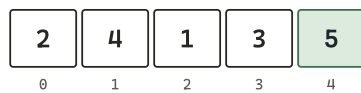
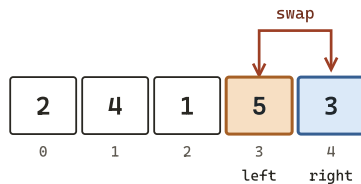
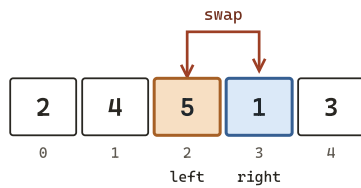
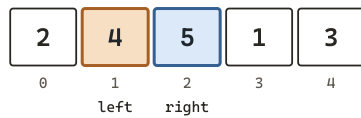
Follow the neighbour checks from left to right. Swap only when the larger value comes first. By the end of a pass, the largest unfinished value has reached its final position on the right.

Green: sorted	Blue: being checked	Orange: position to fill
---------------	---------------------	--------------------------

Pass 1: check the neighbours



Start this pass with [4, 2, 5, 1, 3]. Compare 4 at index 0 with 2 at index 1: $4 > 2$ is true. Swap their positions so the larger value, 4, moves right. The row becomes [2, 4, 5, 1, 3]. Next compare 4 and 5 at indexes 1 and 2.



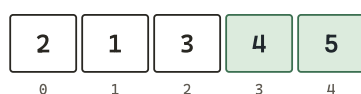
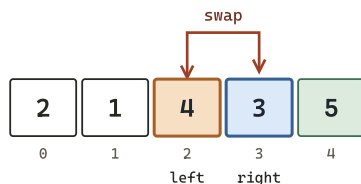
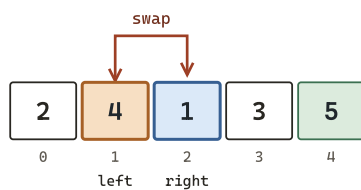
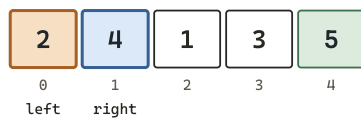
Compare 4 at index 1 with 5 at index 2: $4 > 5$ is false. They are already in order; keep the row $[2, 4, 5, 1, 3]$. Next compare 5 and 1 at indexes 2 and 3.

Compare 5 at index 2 with 1 at index 3: $5 > 1$ is true. Swap their positions so the larger value, 5, moves right. The row becomes $[2, 4, 1, 5, 3]$. Next compare 5 and 3 at indexes 3 and 4.

Compare 5 at index 3 with 3 at index 4: $5 > 3$ is true. Swap their positions so the larger value, 5, moves right. The row becomes $[2, 4, 1, 3, 5]$. That was the last neighbour check in this pass.

The row is $[2, 4, 1, 3, 5]$. The largest unfinished value, 5, is now in its final position at index 4. Next pass, leave indexes 4 through 4 alone. Start again at the left: compare 2 with 4, then continue through index 3.

Pass 2: check the neighbours



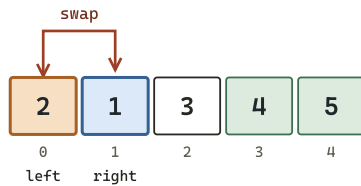
Start this pass with $[2, 4, 1, 3, 5]$. Compare 2 at index 0 with 4 at index 1: $2 > 4$ is false. They are already in order; keep the row $[2, 4, 1, 3, 5]$. Next compare 4 and 1 at indexes 1 and 2.

Compare 4 at index 1 with 1 at index 2: $4 > 1$ is true. Swap their positions so the larger value, 4, moves right. The row becomes $[2, 1, 4, 3, 5]$. Next compare 4 and 3 at indexes 2 and 3.

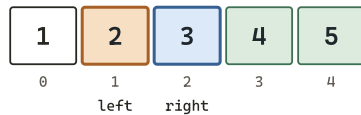
Compare 4 at index 2 with 3 at index 3: $4 > 3$ is true. Swap their positions so the larger value, 4, moves right. The row becomes $[2, 1, 3, 4, 5]$. That was the last neighbour check in this pass.

The row is $[2, 1, 3, 4, 5]$. The largest unfinished value, 4, is now in its final position at index 3. Next pass, leave indexes 3 through 4 alone. Start again at the left: compare 2 with 1, then continue through index 2.

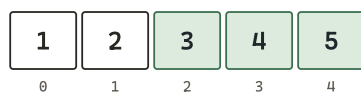
Pass 3: check the neighbours



Start this pass with [2, 1, 3, 4, 5]. Compare 2 at index 0 with 1 at index 1: $2 > 1$ is true. Swap their positions so the larger value, 2, moves right. The row becomes [1, 2, 3, 4, 5]. Next compare 2 and 3 at indexes 1 and 2.

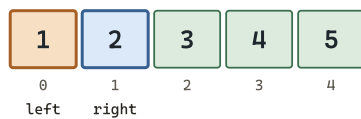


Compare 2 at index 1 with 3 at index 2: $2 > 3$ is false. They are already in order; keep the row [1, 2, 3, 4, 5]. That was the last neighbour check in this pass.



The row is [1, 2, 3, 4, 5]. The largest unfinished value, 3, is now in its final position at index 2. Next pass, leave indexes 2 through 4 alone. Start again at the left: compare 1 with 2, then continue through index 1.

Pass 4: check the neighbours



Start this pass with [1, 2, 3, 4, 5]. Compare 1 at index 0 with 2 at index 1: $1 > 2$ is false. They are already in order; keep the row [1, 2, 3, 4, 5]. That was the last neighbour check in this pass.



The row is [1, 2, 3, 4, 5]. No neighbours needed swapping anywhere in this pass. The row is sorted, so stop.

Sorted: [1, 2, 3, 4, 5]. Each pass settled the largest unfinished value on the right.

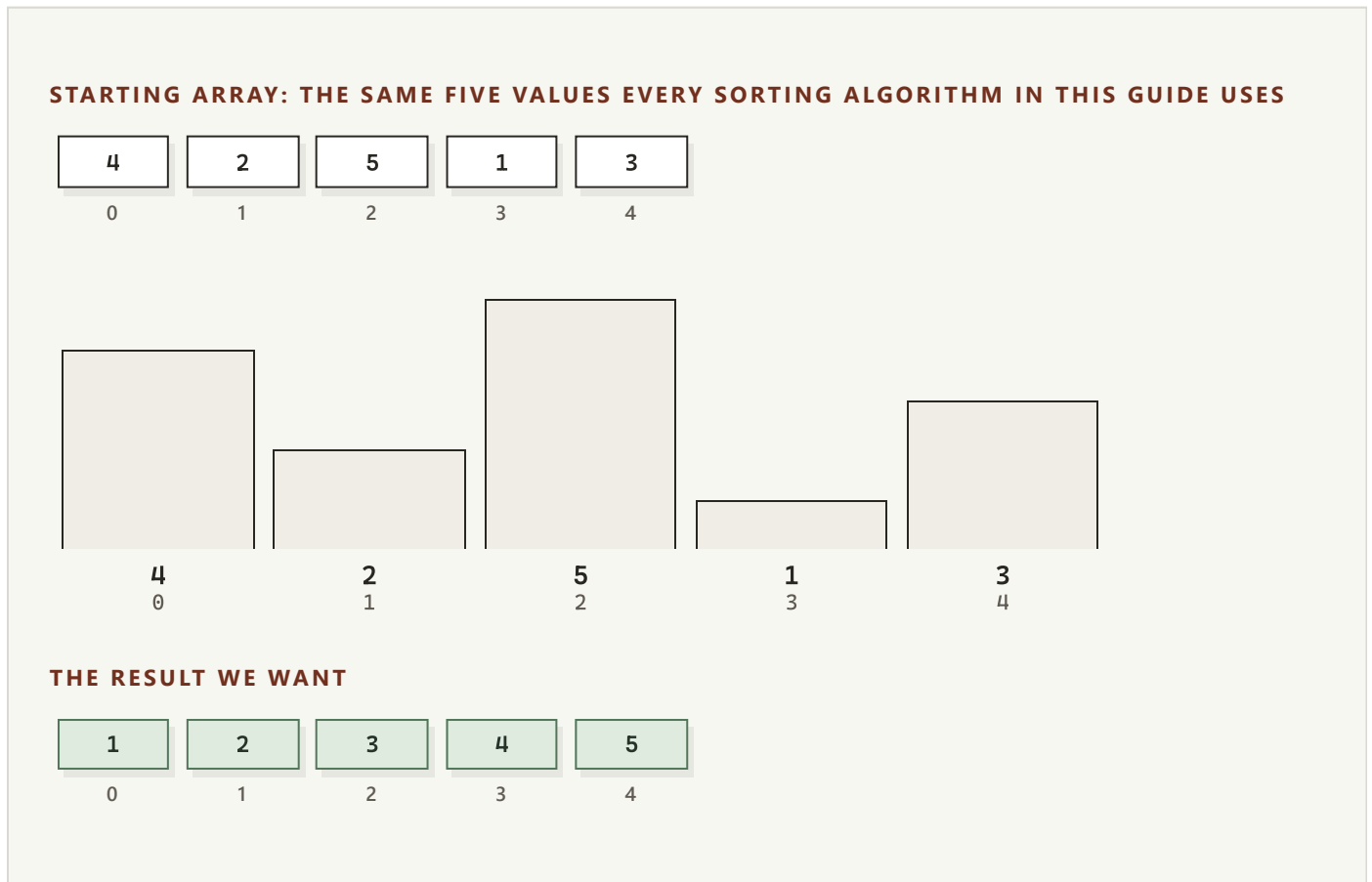
The array we will sort

Detailed reference example: this part introduces its own input, shown below. The opening whole-run map and guided walkthrough keep their original input throughout; each later worked example states its new input explicitly.

Sorting / Bubble sort · The array we will sort

The array we will sort

We will sort `[4, 2, 5, 1, 3]` from smallest to largest, and watch exactly what happens to every value.



Using one array for every algorithm lets you compare them fairly, step against step. The bar picture above shows the same numbers as heights, which is often the quickest way to see what a sorting algorithm is doing: sorting is just turning an uneven skyline into a staircase.

Two details of this array will matter. The 5 is larger than both values after it, so bubble sort will pick it up and carry it to the right end. The 1 is small and sits near the right end: bubble sort can only move it **one place left per pass**, so it takes three passes to reach the front. Keep an eye on both as you read the trace.

Why this works

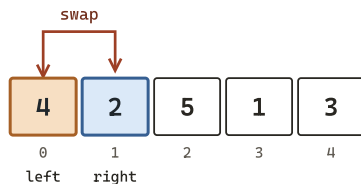
Each neighbour check keeps the larger value on the right. As the pass continues, the largest value reaches the end. Repeat on the unfinished part; stop early if a whole pass makes no swaps.

Guided walkthrough: predict the next major step

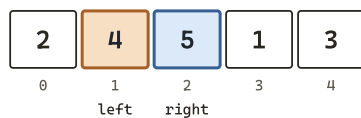
Predict the next completed pass, then advance. Each scene keeps all its movements together.

Guided walkthrough: major steps

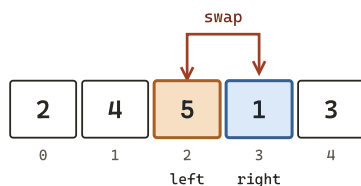
Pass 1: check the neighbours



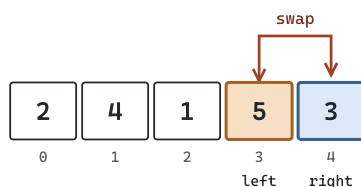
Start this pass with [4, 2, 5, 1, 3]. Compare 4 at index 0 with 2 at index 1: $4 > 2$ is true. Swap their positions so the larger value, 4, moves right. The row becomes [2, 4, 5, 1, 3]. Next compare 4 and 5 at indexes 1 and 2.



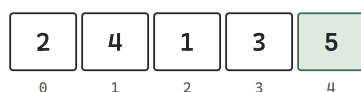
Compare 4 at index 1 with 5 at index 2: $4 > 5$ is false. They are already in order; keep the row [2, 4, 5, 1, 3]. Next compare 5 and 1 at indexes 2 and 3.



Compare 5 at index 2 with 1 at index 3: $5 > 1$ is true. Swap their positions so the larger value, 5, moves right. The row becomes [2, 4, 1, 5, 3]. Next compare 5 and 3 at indexes 3 and 4.



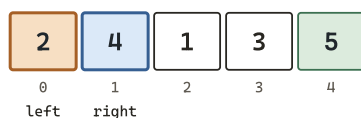
Compare 5 at index 3 with 3 at index 4: $5 > 3$ is true. Swap their positions so the larger value, 5, moves right. The row becomes [2, 4, 1, 3, 5]. That was the last neighbour check in this pass.



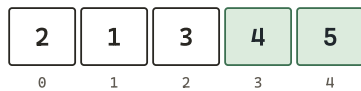
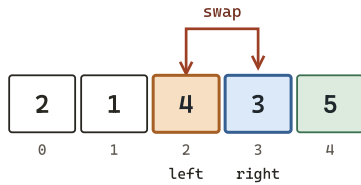
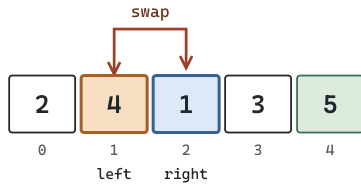
The row is [2, 4, 1, 3, 5]. The largest unfinished value, 5, is now in its final position at index 4. Next pass, leave indexes 4 through 4 alone. Start again at the left: compare 2 with 4, then continue through index 3.

Row after this pass: [2, 4, 1, 3, 5]. Leave the finished positions on the right alone.

Pass 2: check the neighbours



Start this pass with [2, 4, 1, 3, 5]. Compare 2 at index 0 with 4 at index 1: $2 > 4$ is false. They are already in order; keep the row [2, 4, 1, 3, 5]. Next compare 4 and 1 at indexes 1 and 2.



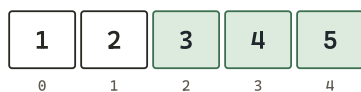
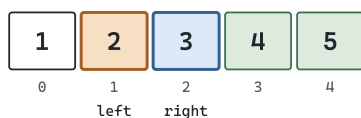
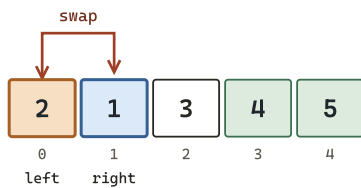
Compare 4 at index 1 with 1 at index 2: $4 > 1$ is true. Swap their positions so the larger value, 4, moves right. The row becomes [2, 1, 4, 3, 5]. Next compare 4 and 3 at indexes 2 and 3.

Compare 4 at index 2 with 3 at index 3: $4 > 3$ is true. Swap their positions so the larger value, 4, moves right. The row becomes [2, 1, 3, 4, 5]. That was the last neighbour check in this pass.

The row is [2, 1, 3, 4, 5]. The largest unfinished value, 4, is now in its final position at index 3. Next pass, leave indexes 3 through 4 alone. Start again at the left: compare 2 with 1, then continue through index 2.

Row after this pass: [2, 1, 3, 4, 5]. Leave the finished positions on the right alone.

Pass 3: check the neighbours



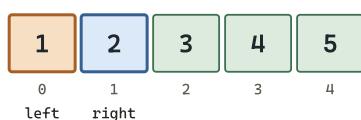
Start this pass with [2, 1, 3, 4, 5]. Compare 2 at index 0 with 1 at index 1: $2 > 1$ is true. Swap their positions so the larger value, 2, moves right. The row becomes [1, 2, 3, 4, 5]. Next compare 2 and 3 at indexes 1 and 2.

Compare 2 at index 1 with 3 at index 2: $2 > 3$ is false. They are already in order; keep the row [1, 2, 3, 4, 5]. That was the last neighbour check in this pass.

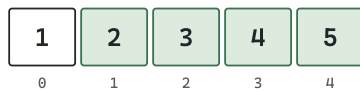
The row is [1, 2, 3, 4, 5]. The largest unfinished value, 3, is now in its final position at index 2. Next pass, leave indexes 2 through 4 alone. Start again at the left: compare 1 with 2, then continue through index 1.

Row after this pass: [1, 2, 3, 4, 5]. Leave the finished positions on the right alone.

Pass 4: check the neighbours



Start this pass with [1, 2, 3, 4, 5]. Compare 1 at index 0 with 2 at index 1: $1 > 2$ is false. They are already in order; keep the row [1, 2, 3, 4, 5]. That was the last neighbour check in this pass.



The row is [1, 2, 3, 4, 5]. No neighbours needed swapping anywhere in this pass. The row is sorted, so stop.

Row after this pass: [1, 2, 3, 4, 5]. No swaps were needed, so sorting is complete.

▼ Full trace table - select a row to jump to its scene

PASS	COMPARISONS	SWAPS	ARRAY AFTER PASS
1	4	3	[2, 4, 1, 3, 5]
2	3	2	[2, 1, 3, 4, 5]
3	2	1	[1, 2, 3, 4, 5]
4	1	0	[1, 2, 3, 4, 5]

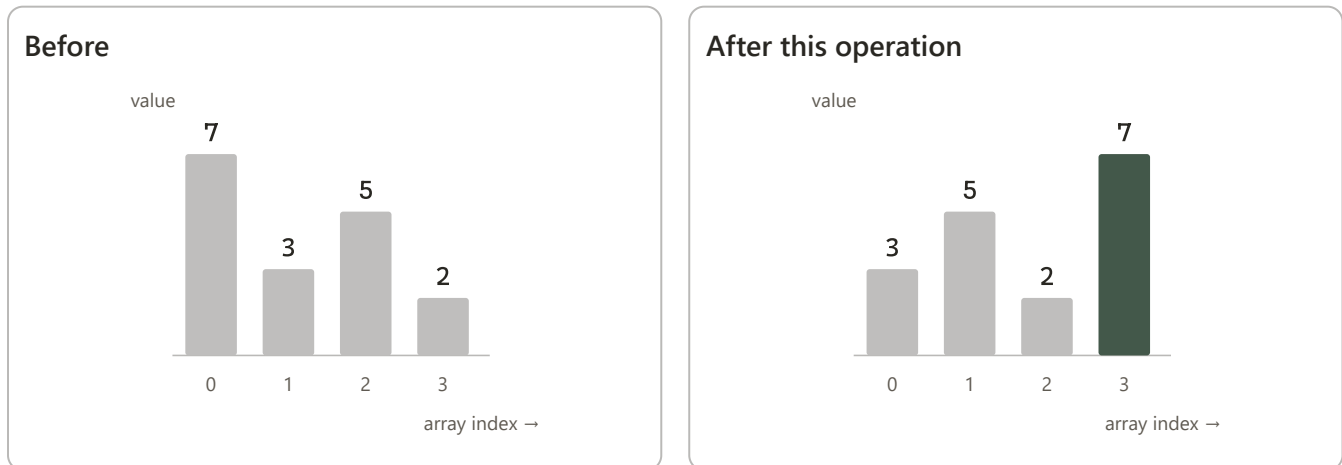
The trace at a glance

PASS	COMPARISONS	SWAPS	ARRAY AFTER PASS
1	4	3	[2, 4, 1, 3, 5]
2	3	2	[2, 1, 3, 4, 5]
3	2	1	[1, 2, 3, 4, 5]
4	1	0	[1, 2, 3, 4, 5]

Read the algorithm as a bar graph

Bar height means **value**; horizontal position means **index**. All panels below use the same height scale. A taller bar is not “more sorted”.

A tall bar travels through adjacent swaps



Compare 7 with 3, then 7 with 5, then 7 with 2. Three adjacent swaps move 7 to index 3. The remaining bars are NOT sorted yet: 5 still precedes 2. One pass fixes the largest active value, not the entire array.

Check yourself: identify which indexes are guaranteed final, which values were merely inspected, and whether the operation swapped, shifted or copied data. A sorted ascending array has non-decreasing heights, including equal-height neighbours.

Fully annotated Java: connect each line to the movement

```
java

public static void bubbleSort(int[] arr) {
    // After pass p, the largest p+1 values are fixed on the right.
    for (int pass = 0; pass < arr.length - 1; pass++) {
        boolean swapped = false; // Reset for THIS pass.
        // Skip the sorted suffix; j+1 remains a valid index.
        for (int j = 0; j < arr.length - 1 - pass; j++) {
            // Strict comparison leaves equal values in their original order.
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j]; // Save before overwriting.
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        // A whole pass with no swap proves every adjacent pair is ordered.
        if (!swapped) break;
    }
}
```

Write it from scratch

- Start a new pass and reset swapped.
- Compare neighbours within the active prefix.
- Swap strictly inverted pairs.
- Shorten the next pass by one fixed position.
- Stop after a whole no-swap pass.

Topic quiz: try it yourself

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What is the last valid index of an array of length 6?

- ☐ A. 5
- ☐ B. 6
- ☐ C. 7

2. What does $O(1)$ auxiliary space describe?

- ☐ A. Exactly one machine instruction
- ☐ B. The input contains one value
- ☐ C. Extra storage stays bounded as n grows

3. What is a loop invariant?

- ☐ A. A loop that never terminates
- ☐ B. A statement that remains true at a specified loop boundary
- ☐ C. A variable that never changes

4. What does stable sorting preserve?

- ☐ A. The original relative order of records with equal keys
- ☐ B. Every original array index
- ☐ C. The original order of unequal keys

5. What should a correct ascending sort do with `[]` and `[7]`?

- ☐ A. Throw an error for both
- ☐ B. Add a zero to each
- ☐ C. Leave both unchanged

6. Which values are compared?

- ☐ A. Every value with one held key
- ☐ B. Neighbours $\text{arr}[j]$ and $\text{arr}[j + 1]$
- ☐ C. Only the first and last values

7. What becomes final after one ascending pass?

- ☐ A. The largest active value at the right edge
- ☐ B. The smallest value at index 0
- ☐ C. The entire array in all cases

8. When may an optimized bubble sort stop early?

- ☐ A. After one ordered pair
- ☐ B. After any one swap
- ☐ C. After an entire pass with no swaps

9. When must swapped be reset to false?

- ☐ A. After every individual comparison
- ☐ B. Before every pass
- ☐ C. Only before the first pass

10. Opening map: array after the first pass?

- ☐ A. [2, 4, 1, 3, 5]
- ☐ B. [1, 2, 3, 4, 5]
- ☐ C. [1, 2, 5, 4, 3]

11. How many input-array writes does a normal swap make?

☐ A. 1

☐ B. 3

☐ C. 2

12. Best-case time with the whole-pass early-stop flag?

☐ A. $O(n^2)$

☐ B. $O(n)$

☐ C. $O(1)$

▼ Answer key and explanations

1. **5.** Indexes start at 0, so the last index is $\text{length} - 1$.
2. **Extra storage stays bounded as n grows.** Auxiliary space counts extra working storage, excluding the input array.
3. **A statement that remains true at a specified loop boundary.** The invariant explains why each iteration preserves progress toward a correct result.
4. **The original relative order of records with equal keys.** Tags such as 2a and 2b let you check whether equal-key records crossed.
5. **Leave both unchanged.** Both inputs are already sorted; no key comparison is necessary.
6. **Neighbours $\text{arr}[j]$ and $\text{arr}[j + 1]$.** Adjacent inversions are exchanged as the scan moves left to right.
7. **The largest active value at the right edge.** Successive adjacent swaps carry a large value right through the active region.
8. **After an entire pass with no swaps.** Only a complete unchanged pass proves all adjacent pairs in the active range ordered.
9. **Before every pass.** The flag describes changes in the current pass only.
10. **[2, 4, 1, 3, 5].** 4 swaps with 2; 5 then swaps with 1 and 3, reaching the rightmost position.
11. **2.** The temporary-variable assignment is not an array write. The two array assignments are.
12. **$O(n)$.** An already sorted array still needs $n - 1$ adjacent comparisons to prove no swaps needed.

The textbook definition

Sorting / Bubble sort · The textbook definition

The textbook definition

Bubble sort is a comparison-based sorting algorithm that makes repeated passes over an array. In each pass it compares every pair of neighbouring values and swaps the pair if it is in the wrong order. After

each pass the largest value in the part still being sorted has moved to its final place at the right end. The algorithm stops when a pass makes no swaps, or when only one value is left.

Pass

One complete walk from the left of the active range to its right end. A pass is not one comparison: it is many.

Adjacent pair

Two neighbouring cells, `array[j]` and `array[j + 1]`. Bubble sort only ever compares neighbours.

Swap

Exchange the two values of a pair so the smaller is on the left (for ascending order).

Active range

The cells a pass still has to look at. It shrinks by one cell after every pass.

Locked tail

The right-hand cells that already hold their final values, so later passes skip them.

swapped flag

A boolean that becomes true when a pass swaps anything. If it stays false, the array is sorted and we can stop early.

Inversion

A pair of values that is in the wrong order. Every swap of neighbours fixes exactly one inversion.

The name comes from the picture: big values slowly “bubble up” to the end of the array the way air bubbles rise through water, while small values sink towards the front, only one place at a time.

In plain language

Sorting / Bubble sort · In plain language

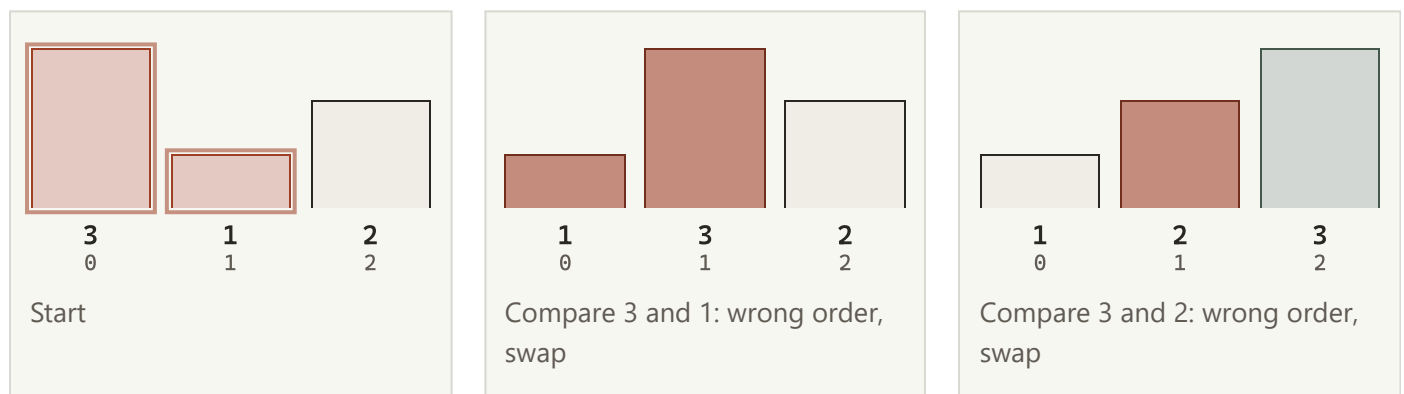
In plain language

Think of walking along a line of people and swapping any neighbours who are standing in the wrong order.

Picture a row of students you want ordered from shortest to tallest. You stand at the left end and look at the first two students. If the left one is taller, they swap places. Then you take one step right and look at the next pair, and so on to the end of the row.

When you reach the end, something useful has happened without any planning: **the tallest student has been swapped all the way to the right end**, because whenever you met a taller student you carried them one step further. Walk along again (stopping one place earlier, since the end is finished) and the second tallest lands next to them.

Here is the smallest possible example, three values [3, 1, 2], drawn as bars. The two bars that are being compared are outlined; the bars that just changed places are filled darker.



being compared just swapped final position

1 3 > 1 **TRUE · SWAP**

2 3 > 2 **TRUE · SWAP**

That was pass 1: two comparisons, two swaps, and the 3 is home. Pass 2 compares only 1 and 2; they are already in order, nothing is swapped, and the algorithm knows it is finished.

The main idea

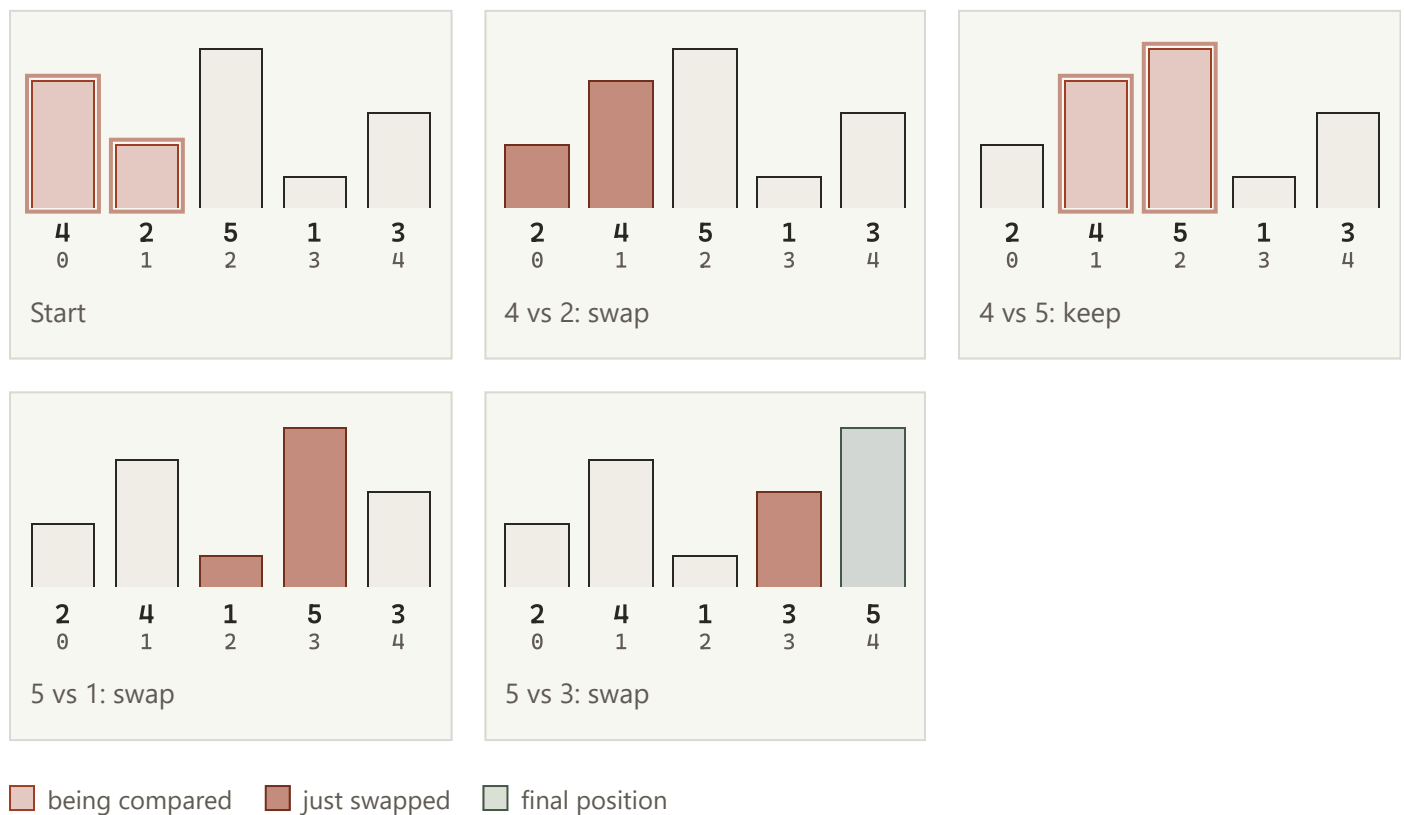
Sorting / Bubble sort · The main idea

The main idea

Bubble sort never looks at the whole array at once. It follows one tiny rule, “if the left neighbour is bigger, swap”, and repeats it along the array. Everything else follows from that rule.

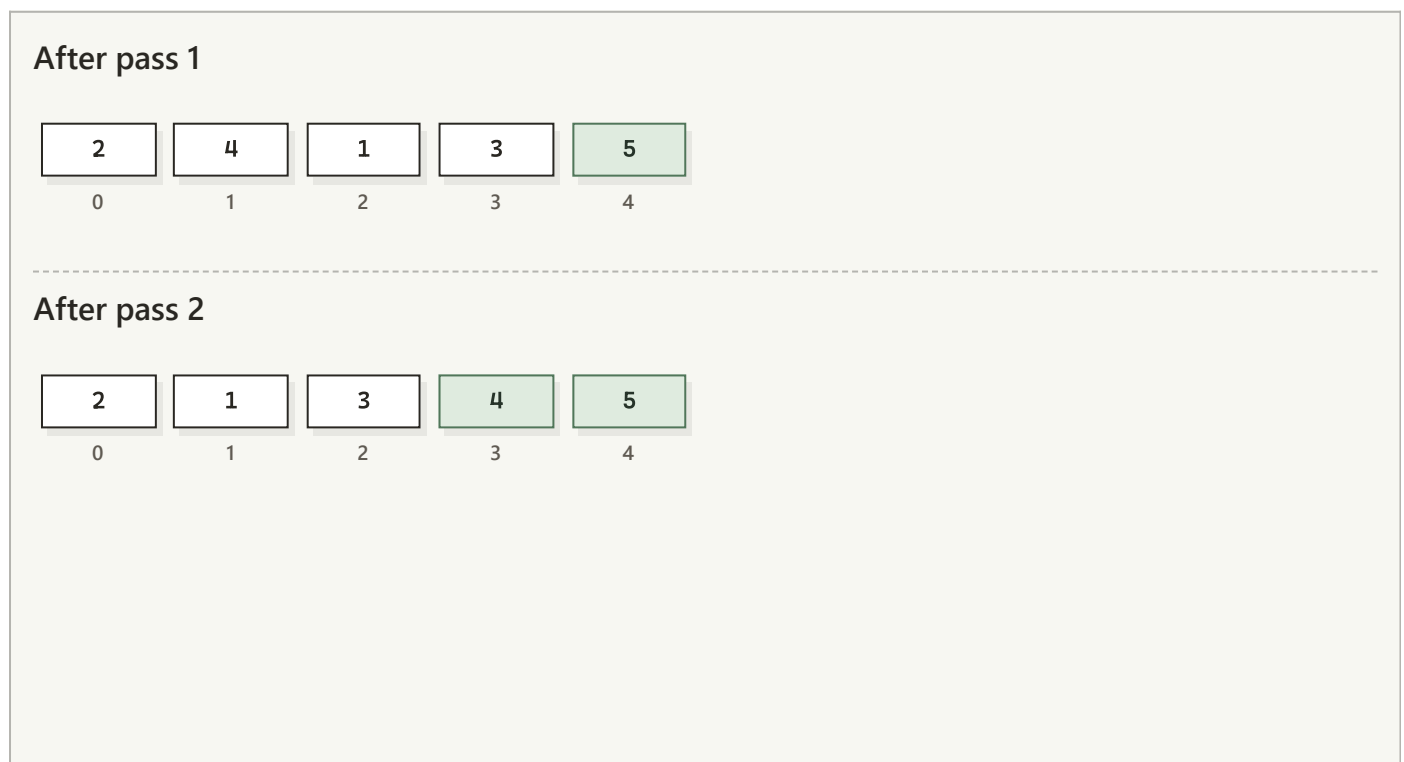
One pass carries the biggest value to the end

Watch pass 1 on [4, 2, 5, 1, 3]. The 4 is carried one step (it beats 2), then meets 5 and is left behind. From then on it is the 5 that gets carried, past 1 and past 3, all the way to the end. A larger value is only stopped by an even larger one, so the maximum can never be stopped.

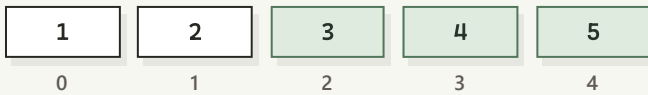


After pass k , the last k cells are final

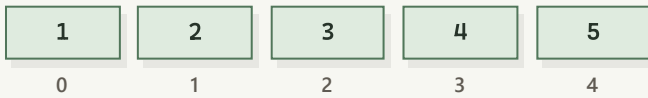
The maximum reaches the last cell, the next pass brings the second largest to the cell before it, and so on. Cells in the **locked tail** never need to be compared again, which is why each pass can stop one cell earlier than the one before.



After pass 3



After pass 4



Small values move slowly

Notice the 1: it starts at index 3 and moves left **one place per pass** (index 2, then 1, then 0). A big value can travel the whole array in one pass; a small value can only creep. That asymmetry is the reason bubble sort is slow on some inputs, and we will measure it in the examples.

The repeating recipe: compare a pair → swap if the left one is larger → step right → at the end of the range lock the last cell → start the next pass, one cell shorter → stop when a pass makes no swaps.

Comparison, swap, pass

Sorting / Bubble sort · Comparison, swap, pass

Comparison, swap, pass

These words describe different things. Counting them separately is what makes a trace table correct, and exam questions often ask for exactly one of them.

Comparison

One question, such as $4 > 2$. Bubble sort asks one for every pair in every pass, whether or not anything moves.

Swap

Exchanging two neighbours. It happens only when the answer to the comparison was *yes, they are out of order*.

Pass

A whole run of comparisons along the active range. “Show the array after each pass” means one snapshot per pass, not per swap.

Early exit

Stopping because a pass made no swaps. Without the flag, bubble sort keeps making every pass even on sorted data.

How one swap actually works

Java cannot exchange two cells in a single step. If you wrote `a[j] = a[j + 1]; a[j + 1] = a[j];` both cells would end up equal, because the first line destroys the original `a[j]`. A third variable, `temp`, keeps it safe. Follow the swap of `a[j] = 4` and `a[j + 1] = 2`:

STEP	CODE	TEMP	A[J]	A[J + 1]	WHAT HAPPENED
0	(before)	—	4	2	Out of order, so we swap
1	<code>int temp = a[j];</code>	4	4	2	The 4 is saved
2	<code>a[j] = a[j + 1];</code>	4	2	2	Both cells hold 2; the 4 survives only in <code>temp</code>
3	<code>a[j + 1] = temp;</code>	4	2	4	The 4 is restored on the right: swapped

Swap versus shift

Insertion sort *shifts*: it copies a larger value one place right while the key waits in a variable, then drops the key in once. Bubble sort *swaps*: it exchanges two neighbours using three assignments. They fix the same kind of problem (an out-of-order pair) but a swap does more writing, which is one reason insertion sort is usually the faster of the two in practice.

Every swap fixes exactly one inversion

An **inversion** is a pair of values in the wrong order. In `[4, 2, 5, 1, 3]` the inversions are (4,2), (4,1), (4,3), (2,1), (5,1), (5,3): **6** of them. A swap of neighbours repairs exactly one pair and breaks none, so bubble sort needs exactly **6 swaps** for this array, and that is what the trace shows. Counting inversions is a quick way to predict the number of swaps before you trace anything.

Compare neighbouring values

Sorting / Bubble sort

Compare neighbouring values

Bubble sort repeatedly compares adjacent values. When the left value is larger, the pair swaps. After one complete pass, the largest unsorted value has moved to the end.

Illustration · first pass

Start with the lecture array `[63, 29, 72, 85, 18]`.

1

63 > 29

SWAP → [29,63,72,85,18]

2

63 > 72

NO SWAP

3

72 > 85

NO SWAP

4

85 > 18

SWAP → [29,63,72,18,85]



Trace table

PASS	ARRAY AFTER PASS	COMPARISONS	SWAPS	GUARANTEED RESULT
1	[29,63,72,18,85]	4	2	85 is fixed
2	[29,63,18,72,85]	3	1	72 is fixed
3	[29,18,63,72,85]	2	1	63 is fixed
4	[18,29,63,72,85]	1	1	Array sorted

Implementation

● ● ● Optimised bubble sort

```
public static void bubbleSort(int[] arr) {
    // Each pass locks one more large value at the right.
    for (int pass = 0; pass < arr.length - 1; pass++) {
        boolean swapped = false;
        // Ignore the positions already locked by earlier passes.
        for (int j = 0; j < arr.length - 1 - pass; j++) {
            // Ascending order: swap an inverted neighbouring pair.
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }
        // No swaps means the whole array is already sorted.
        if (!swapped) break;
    }
}
```

▼ Recursive bubble sort

First call: `bubbleSortRec(arr, arr.length)`.

● ● ● Java

```
public static void bubbleSortRec(int[] arr, int n) {
    // Base case: zero or one active value is sorted.
    if (n ≤ 1) return;

    boolean swapped = false;
    // One pass moves the largest active value to index n - 1.
    for (int j = 0; j < n - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
            swapped = true;
        }
    }
    // Sort the remaining prefix only if the pass changed it.
    if (swapped) bubbleSortRec(arr, n - 1);
}
```

Write it from scratch

- 1 Repeat passes.
- 2 Compare indexes j and $j + 1$.
- 3 Swap an inverted pair.
- 4 Shorten the next pass.
- 5 Stop early after a no-swap pass.

Ascending ↔ descending

Ascending swaps when `arr[j] > arr[j+1]`.

Descending swaps when `arr[j] < arr[j+1]`.

Same pair, opposite decision: for neighbours 63, 29, ascending swaps ($63 > 29$) while descending keeps them ($63 < 29$ is false). In every descending pass the *smallest* active value is carried to the right edge.

Debug checklist

- Stop at $n - 1 - \text{pass}$.
- Reset `swapped` for every pass.
- Use a temporary variable for the swap.

Worked example · full run on [4, 2, 5, 1, 3]

This is the same array the insertion-sort chapter uses, so the work can be compared. The orange line marks the boundary of the values already locked at the right.

Pass 1 · compare neighbours with $j = 0..3$ (pairs within cells 0–4) · 4 comparisons, 3 swaps

4	2	5	1	3
0	1	2	3	4

1 4 > 2 **SWAP → [2, 4, 5, 1, 3]**

2 4 > 5 **NO SWAP**

3 5 > 1 **SWAP → [2, 4, 1, 5, 3]**

4 5 > 3 **SWAP → [2, 4, 1, 3, 5]**

2	4	1	3	5
0	1	2	3	4 LOCKED

After pass 1, 5 is fixed at index 4.

Pass 2 · compare neighbours with $j = 0..2$ (pairs within cells 0–3) · 3 comparisons, 2 swaps

2	4	1	3	5
0	1	2	3	4

1 2 > 4 **NO SWAP**

2 4 > 1 **SWAP → [2, 1, 4, 3, 5]**

3 4 > 3 **SWAP → [2, 1, 3, 4, 5]**

2	1	3	4	5
0	1	2	3 LOCKED	4

After pass 2, 4 is fixed at index 3.

Pass 3 · compare neighbours with j = 0..1 (pairs within cells 0–2) · 2 comparisons, 1 swap



1 2 > 1 **SWAP → [1, 2, 3, 4, 5]**

2 2 > 3 **NO SWAP**



After pass 3, 3 is fixed at index 2.

Pass 4 · compare neighbours with j = 0 only (cells 0–1) · 1 comparison, 0 swaps



1 1 > 2 **NO SWAP**



After pass 4, 2 is fixed at index 1.

PASS	ARRAY AFTER PASS	COMPARISONS	SWAPS	GUARANTEED
1	[2, 4, 1, 3, 5]	4	3	5 fixed
2	[2, 1, 3, 4, 5]	3	2	4 fixed
3	[1, 2, 3, 4, 5]	2	1	3 fixed
4	[1, 2, 3, 4, 5]	1	0	Array sorted
Total	—	10	6	Sorted

Sorted after 4 passes: 10 comparisons and 6 swaps. The 6 swaps equal the 6 inversions in the starting array.

Best case · already sorted

For [1, 2, 3, 4], show every neighbour comparison in the first pass.

Pass 1 · active range 0..3

1

2

3

4

0

1

2

3

1

1 > 2

FALSE · NO SWAP

2

2 > 3

FALSE · NO SWAP

3

3 > 4

FALSE · NO SWAP

swapped == false → stop after one pass. Best time: O(n).

PASS	BEFORE	COMPARISONS	SWAPS	AFTER	DECISION
1	[1, 2, 3, 4]	3	0	[1, 2, 3, 4]	Stop early

BEST

$O(n)^*$

AVERAGE

$O(n^2)$

WORST

$O(n^2)$

STABLE

Yes

IN-PLACE

Yes

ADAPTIVE

Yes*

ITERATIVE SPACE

$O(1)$

RECURSIVE SPACE

$O(n)$

* Best case $O(n)$ and “adaptive” apply to the optimised version shown here, which stops after a pass with no swaps. Without the swapped flag, bubble sort always makes $n(n - 1) / 2$ comparisons.

Questions

▼ What does one full pass guarantee?

The largest value in the unsorted section reaches its final position at the right.

▼ Why does the inner loop become shorter?

Values at the right have already been locked into their final positions, so comparing them again would be wasted work.

Bubble sort in one sentence

Swap inverted neighbours over repeated passes so the largest active value bubbles into its final position on the right.

Recursion, calls and returns

java

```
public static void bubbleSortRec(int[] arr, int n) {
    // n is the active prefix LENGTH, not an index.
    if (n ≤ 1) return;
    boolean swapped = false;
    for (int j = 0; j < n - 1; j++) {
        if (arr[j] > arr[j + 1]) {
            int temp = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = temp;
            swapped = true;
        }
    }
    // The largest active value is now at n-1; shrink the prefix.
    if (swapped) bubbleSortRec(arr, n - 1);
    // First call: bubbleSortRec(arr, arr.length).
}
```

CALL STACK - NEWEST FRAME AT THE TOP

bubbleSortRec(n=4)

Pass fixes the largest remaining value at index 3

bubbleSortRec(n=5)

Waiting: Child sorts remaining prefix 0..3

Unlike iterative auxiliary space, suspended recursive frames require $O(n)$ stack space in the worst case.

A complete runnable Java example

Save as `BubbleDemo.java`. The code includes the method(s) and a main entry point.

```
import java.util.Arrays;

public class BubbleDemo {
    public static void bubbleSort(int[] arr) {
        // After pass p, the largest p+1 values are fixed on the right.
        for (int pass = 0; pass < arr.length - 1; pass++) {
            boolean swapped = false; // Reset for THIS pass.
            // Skip the sorted suffix; j+1 remains a valid index.
            for (int j = 0; j < arr.length - 1 - pass; j++) {
                // Strict comparison leaves equal values in their original order.
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j]; // Save before overwriting.
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }
            // A whole pass with no swap proves every adjacent pair is ordered.
            if (!swapped) break;
        }
    }

    public static void main(String[] args) {
        int[] values = {4, 2, 5, 1, 3};
        bubbleSort(values);
        System.out.println(Arrays.toString(values));
    }
}
```

```
[1, 2, 3, 4, 5]
```

Properties

Sorting / Bubble sort · Properties

Properties

BEST TIME $O(n)^*$	AVERAGE TIME $O(n^2)$
WORST TIME $O(n^2)$	EXTRA SPACE $O(1)$
STABLE Yes	IN-PLACE Yes
ADAPTIVE Yes*	TYPE Comparison

* only with the swapped early-exit flag. Without it, bubble sort always makes $n(n - 1) / 2$ comparisons.

Stable

Equal values keep their original left-to-right order, because a pair is swapped only when the left value is *strictly greater*. Equal neighbours are left alone, so they can never cross each other.

In-place

Everything happens inside the original array. The only extra storage is a few variables (`j`, `pass`, `temp`, `swapped`), so the extra space is constant, $O(1)$.

Adaptive (with the flag)

On sorted or nearly sorted data the flag lets the algorithm stop after very few passes. But note *nearly sorted* must mean “no small value is far to the right”: one turtle forces many passes (see `[2, 3, 4, 5, 1]`).

Comparison-based

All decisions come from asking whether one value is greater than another; it never uses the value's digits or range. This is why no comparison-based sort can beat $O(n \log n)$ in general.

Swaps equal inversions

The number of swaps is exactly the number of inversions in the input, from 0 (sorted) to $n(n - 1) / 2$ (reversed).

Why those complexities occur

Sorting / Bubble sort · Why those complexities occur

Why those complexities occur

Best case · $O(n)$

An already sorted array: pass 1 makes $n - 1$ comparisons, finds no swap and stops.

Worst case · $O(n^2)$

A reversed array: every pair is out of order in every pass, so all $n(n - 1) / 2$ comparisons also swap.

Counting the comparisons

Pass 1 compares $n - 1$ pairs. Pass 2 compares one fewer, because the last cell is locked. Pass 3 one fewer again, down to a final pass of one comparison. For five values:

pass 1		4
pass 2		3
pass 3		2
pass 4		1

total = $4 + 3 + 2 + 1 = 10$

Each row is one pass and each square one comparison. The shape is a staircase, and the area of a staircase of height $n - 1$ is half of a square:

$$(n - 1) + (n - 2) + \dots + 2 + 1 = n(n - 1) / 2 \rightarrow O(n^2)$$

For $n = 8$ the staircase has 28 squares ($7 + 6 + 5 + 4 + 3 + 2 + 1$). Here is how fast that grows:

N (VALUES)	COMPARISONS $N(N - 1) / 2$
5	10
10	45
100	4,950
1,000	499,500
2,000	1,999,000

Double the array from 1,000 to 2,000 values and the work does not double, it roughly **quadruples** ($499,500 \rightarrow 1,999,000$). That is what “quadratic” means in practice, and it is why bubble sort is only comfortable for small arrays.

Best case with the flag

For [1, 2, 3, 4, 5] the first pass makes 4 comparisons, sets nothing, and the loop ends: $n - 1$ steps, so $O(n)$. Without the **swapped** flag the staircase would still be walked in full.

Average case

For random data about half of all pairs are inverted, so about $n(n - 1) / 4$ swaps happen. The constant halves, but the growth is still proportional to n^2 .

Space

Nothing is copied. A few variables are enough, so the space is $O(1)$.

When to use it

Sorting / Bubble sort · When to use it

When to use it

A good choice

Teaching the idea of sorting; tiny arrays (a handful of values); checking whether data is already sorted (one pass with no swaps proves it); data where the only disorder is a few *large* values near the front.

A poor choice

Anything large or random: $O(n^2)$ comparisons grow too fast; data with a small value far to the right (turtles); any situation where you need guaranteed $O(n \log n)$ time; and almost every real program, where insertion sort or merge sort does better.

In practice bubble sort is a learning tool, not a working tool. It earns its place because it is the clearest example of *repeated local fixing*: a rule about two neighbours, applied repeatedly, produces a globally sorted array. When you are asked in an exam to choose between algorithms for a large array, bubble sort is almost never the answer.

Compared with its two quadratic cousins: **insertion sort** does less writing and is better on nearly sorted data; **selection sort** makes the fewest swaps. Bubble sort is the one that can notice “already sorted” in a single pass, but only if you remember the flag.

Why sort data at all?

Sorting · Why sort data at all?

Why sort data at all?

Before the algorithms: what does sorted data give you that unsorted data does not, and what does it cost?

Everything in this chapter is about putting values in order. Before learning *how*, it is worth being clear about **why**, because it is a favourite exam question and because it explains why searching and sorting are taught together.

Short answer for a test: sorting data makes it easier and faster to find information in it. A sorted array can be searched with binary search, in $O(\log n)$ instead of $O(n)$.

What becomes easy once data is sorted

Take the nine values from the past paper. Unsorted they tell you nothing at a glance; sorted, a lot of questions answer themselves.



QUESTION	UNSORTED ARRAY	SORTED ARRAY
Smallest and largest value?	Look at all 9 cells	The first and last cell: 2 and 90
Median (middle) value?	Needs a lot of work	The middle cell: 50
Is a value present?	Linear search: up to 9 comparisons	Binary search: at most 4 probes
Where are the duplicates?	Anywhere	Next to each other (see below)
Merge two lists?	No shortcut	Merge them by zipping (that is merge sort's merge step)
Show a list to a person?	Hard to read	Alphabetical or numerical order, easy to scan

Duplicates end up side by side

Finding repeated values in [3, 1, 3, 2, 3] means comparing every pair. After sorting, [1, 2, 3, 3, 3], equal values are neighbours, so one pass comparing each cell with the next finds them all.

Sorting is not free: is it worth it?

Sorting costs about $n \log_2 n$ steps, and it only pays off if you search more than a few times. Suppose you have **1,000,000** values and will search for **1,000** different keys:

STRATEGY	WORK	TOTAL
Never sort: linear search every time	1,000 searches × up to 1,000,000 comparisons	1,000,000,000
Sort once, then binary search every time	sort ≈ 20,000,000 steps, then 1,000 searches × 20 probes	20,020,000

That is about **49× less work** after sorting. With a single search the answer flips: scanning once costs 1,000,000 and sorting first costs 20,000,000. The rule: *search once, scan; search often, sort first*.

▼ Quick check: Exam-style question 3(a), “What is the purpose of sorting data?”

To arrange data in a defined order (ascending or descending) so that information is easier and faster to find, for example by using binary search instead of scanning every element. Other valid points: duplicates become adjacent, minimum and maximum are at the ends, and sorted data is easier to read and merge.

The sorting map

Sorting

The sorting map

Every algorithm orders values, but each uses a different repeated action. The four are taught in this order: Bubble, Selection, Insertion, Merge.

ALGORITHM	REPEATED ACTION	TYPICAL TIME	BEST FIT
Bubble	Compare neighbours; swap wrong pairs	$O(n^2)$	Learning and tiny inputs
Selection	Find the smallest remaining value; place it next	$O(n^2)$	When few swaps matter
Insertion	Shift larger sorted values; insert a key	$O(n^2)$	Small or nearly sorted data
Merge	Divide into halves; merge sorted halves	$O(n \log n)$	Large data and predictable speed

Vocabulary behind every property tile

Comparison-based Order is decided only by asking questions such as $a < b$.	Best / average / worst The same algorithm on three arrangements of the same n values: already ordered, typical, and the most awkward.
In-place Values are rearranged inside the original array using only a few extra variables: $O(1)$ auxiliary space.	Auxiliary space Extra memory beyond the input array, such as the temporary copies merge sort makes.
Adaptive Does less work when the input is already or nearly sorted.	Stable Equal values keep their original relative order.

Why stability matters

Sort these students by mark. Ana and Ben both scored 70, and Ana was listed first.

Start

70	70	65
ANA	BEN	CY

Stable sort result

65	70	70
CY	ANA	BEN

An unstable sort may give



Both outputs are sorted by mark. Only the stable one keeps Ana before Ben, which matters when you sort by one field and want an earlier ordering by another field preserved.

Count the disorder: inversions

An **inversion** is a pair of positions whose values are in the wrong order: a larger value sits to the left of a smaller one. [4, 2, 5, 1, 3] has six: (4,2), (4,1), (4,3), (2,1), (5,1), (5,3).

ALGORITHM	WHAT IS COUNTED	ON [4, 2, 5, 1, 3]	LINK TO INVERSIONS
Bubble	adjacent swaps	6	Each swap of neighbours removes exactly one inversion.
Insertion	shifts	6	Each shift removes exactly one inversion.
Selection	swaps	2	At most $n - 1$ swaps; one swap can remove several inversions at once.

A fully reversed array of n values has $n(n - 1) / 2$ inversions. For [5, 4, 3, 2, 1] that is 10, which equals the 10 shifts in the reverse-order insertion example. This is the reason bubble and insertion sort are $O(n^2)$ in the worst case.

Descending order

Sorting / Bubble sort · Descending order

Descending order

To sort from largest to smallest, change **one symbol**: swap when the left value is *smaller* than its right neighbour.

Ascending

```
if (arr[j] > arr[j + 1]) {
    swap(arr, j, j + 1);
}
```

Descending

```
if (arr[j] < arr[j + 1]) {
    swap(arr, j, j + 1);
}
```

Nothing else moves: the loops, the shrinking range and the **swapped** flag stay identical. The only difference is which value gets carried. In descending order the *smallest* value of the active range is carried to the right end each pass, and the largest values drift left.

	ASCENDING	DESCENDING
Start	[4, 2, 5, 1, 3]	[4, 2, 5, 1, 3]
After pass 1	[2, 4, 1, 3, 5] · 3 swaps	[4, 5, 2, 3, 1] · 2 swaps
After pass 2	[2, 1, 3, 4, 5] · 2 swaps	[5, 4, 3, 2, 1] · 2 swaps
After pass 3	[1, 2, 3, 4, 5] · 1 swaps	[5, 4, 3, 2, 1] · 0 swaps
After pass 4	[1, 2, 3, 4, 5] · 0 swaps	—

Both runs start from the same array. Ascending needs **6 swaps** (the 6 inversions). Descending needs **4 swaps**: out of the 10 possible pairs, 6 are out of order for ascending, so the other 10 – 6 = 4 pairs are out of order for descending. Comparisons: 10 for ascending and 9 for descending, because the early-exit pass happens at a different moment.

Test habit: when a question says *descending*, write the direction at the top of your working and check the first comparison before you start. Most lost marks on this question type come from silently sorting in the wrong direction.

Worked examples

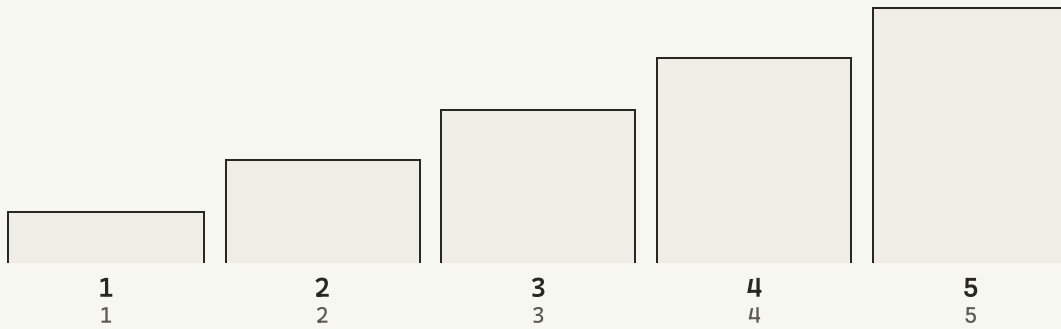
Sorting / Bubble sort · Worked examples

Worked examples

Six arrays, each chosen to teach one thing: the best case, the worst case, fast values, slow values, stability, and a longer realistic run.

Already sorted · [1, 2, 3, 4, 5]

Every neighbouring pair is already in order, so the first pass swaps nothing.



Start



After pass 1 · no swaps, so the array is sorted

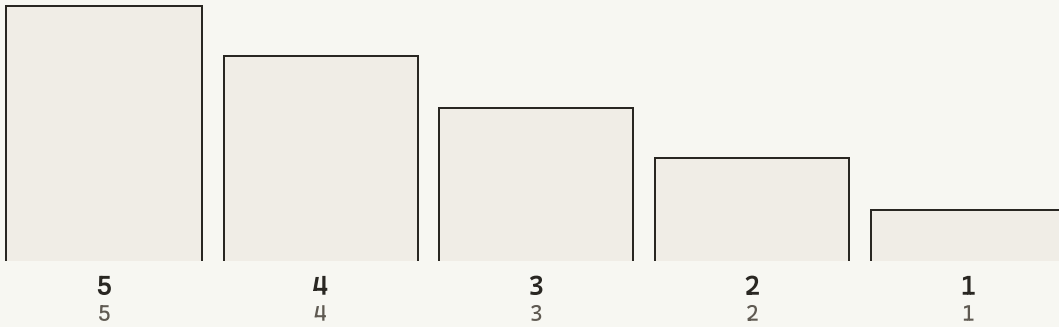


PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[1, 2, 3, 4, 5]	—	—
1	j = 0..3	[1, 2, 3, 4, 5] (<i>no swaps: stop</i>)	4	0
Total		sorted	4	0

4 comparisons and 0 swaps: one pass is enough. This early exit is what makes bubble sort $O(n)$ in the best case, *but only because of the swapped flag*.

Reverse order · [5, 4, 3, 2, 1]

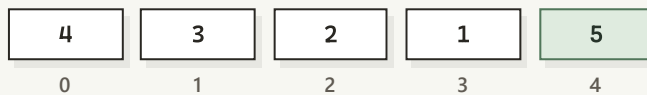
Every pair is out of order, so every comparison swaps.



Start



After pass 1 · the last 1 cell is final



After pass 2 · the last 2 cells are final



After pass 3 · the last 3 cells are final



After pass 4 · the last 4 cells are final



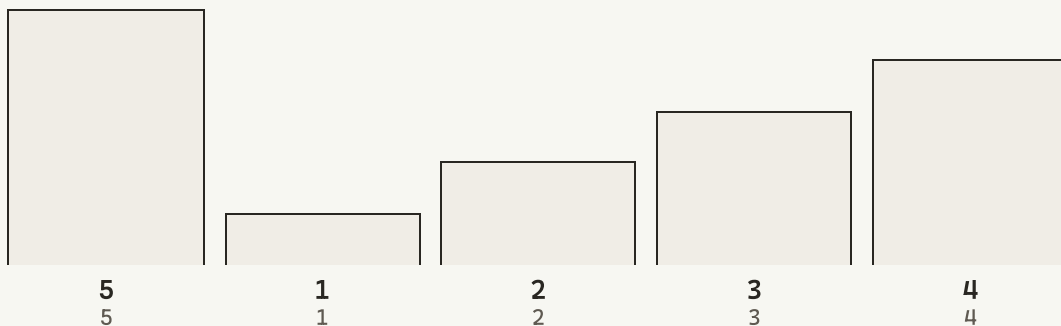
PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[5, 4, 3, 2, 1]	—	—
1	j = 0..3	[4, 3, 2, 1, 5]	4	4

PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
2	j = 0..2	[3, 2, 1, 4, 5]	3	3
3	j = 0..1	[2, 1, 3, 4, 5]	2	2
4	j = 0 only	[1, 2, 3, 4, 5]	1	1
Total		sorted	10	10

10 comparisons and 10 swaps: $1 + 2 + 3 + 4 = n(n - 1) / 2$. Comparisons and swaps are equal here, the worst case.

A large value near the front · [5, 1, 2, 3, 4]

The one out-of-place value is large, so a single pass carries it all the way to the right.



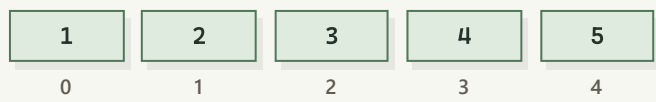
Start

5	1	2	3	4
0	1	2	3	4

After pass 1 · the last 1 cell is final

1	2	3	4	5
0	1	2	3	4

After pass 2 · no swaps, so the array is sorted

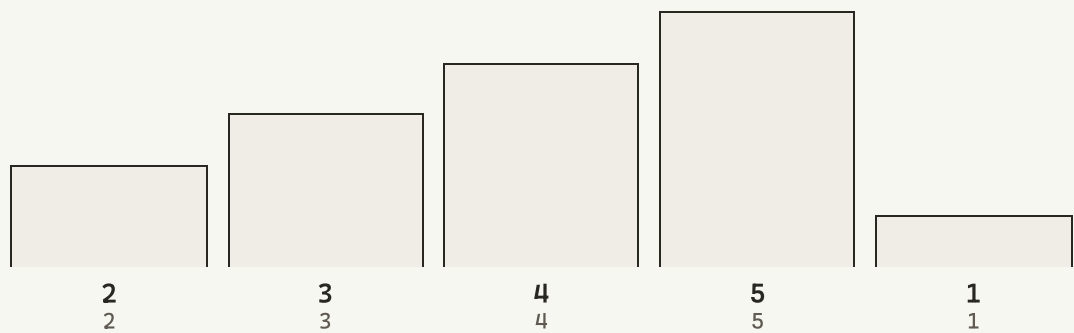


PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[5, 1, 2, 3, 4]	—	—
1	j = 0..3	[1, 2, 3, 4, 5]	4	4
2	j = 0..2	[1, 2, 3, 4, 5] (no swaps: stop)	3	0
Total		sorted	7	4

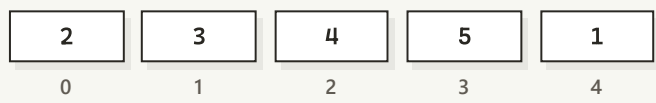
Only 2 passes (the second one just confirms the order), 4 swaps. Large values travel fast: bubble sort calls these *rabbits*.

A small value near the back · [2, 3, 4, 5, 1]

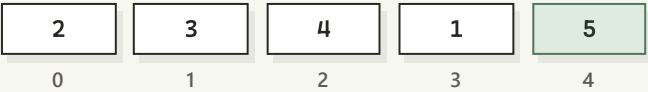
The same five values, but now the out-of-place value is the small one at the end.



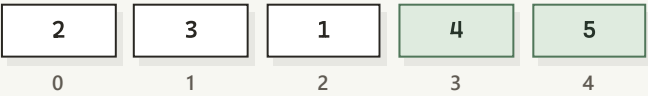
Start



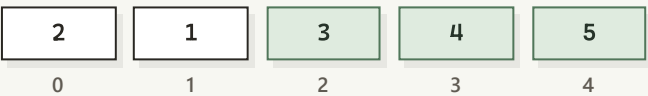
After pass 1 · the last 1 cell is final



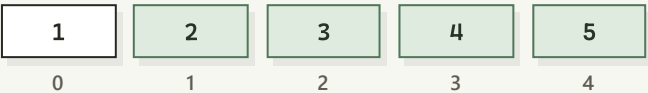
After pass 2 · the last 2 cells are final



After pass 3 · the last 3 cells are final



After pass 4 · the last 4 cells are final

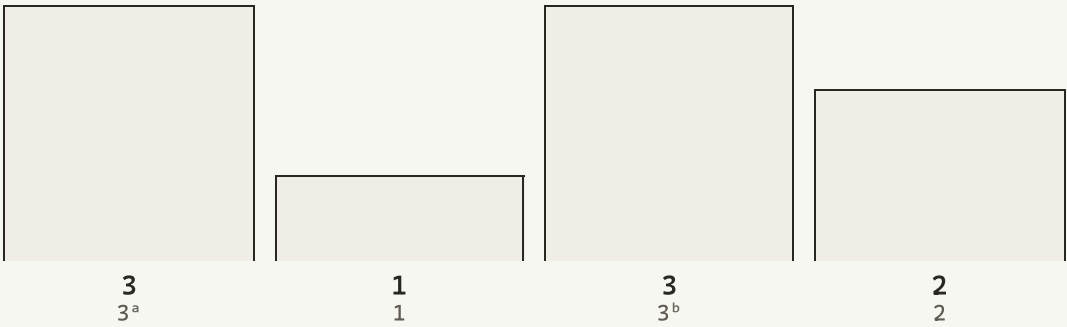


PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[2, 3, 4, 5, 1]	—	—
1	j = 0..3	[2, 3, 4, 1, 5]	4	1
2	j = 0..2	[2, 3, 1, 4, 5]	3	1
3	j = 0..1	[2, 1, 3, 4, 5]	2	1
4	j = 0 only	[1, 2, 3, 4, 5]	1	1
Total		sorted	10	4

4 passes, 4 swaps. The 1 moves one place left per pass, so it needs four passes. Small values near the end are *turtles*, and they are what makes bubble sort slow.

Duplicates · [3^a, 1, 3^b, 2]

Two equal values, labelled so we can tell them apart. The rule is to swap only when the left value is *strictly* larger.



Start



After pass 1 · the last 1 cell is final



After pass 2 · the last 2 cells are final



After pass 3 · no swaps, so the array is sorted



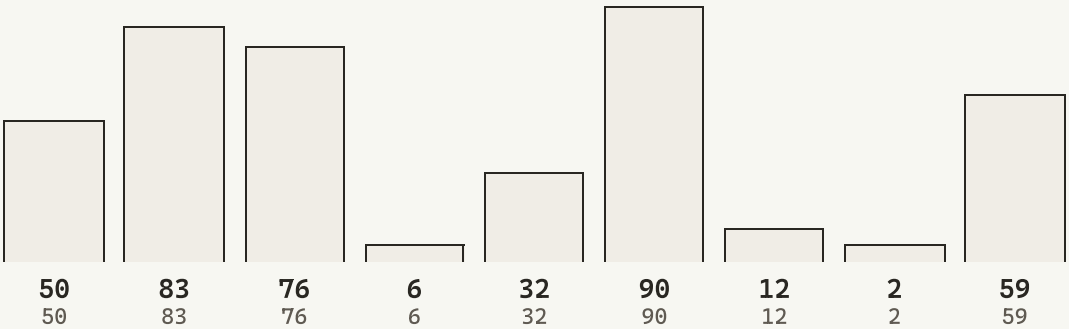
PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[3 ^a , 1, 3 ^b , 2]	—	—
1	j = 0..2	[1, 3 ^a , 2, 3 ^b]	3	2

PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
2	$j = 0..1$	[1, 2, 3 ^a , 3 ^b]	2	1
3	$j = 0$ only	[1, 2, 3 ^a , 3 ^b] (<i>no swaps: stop</i>)	1	0
Total		sorted	6	3

The result is [1, 2, 3^a, 3^b]: 3^a is still before 3^b. Equal neighbours are never swapped, so bubble sort is **stable**. Writing $\geq$ instead of $>$ would swap them and lose that.

A longer array · [50, 83, 76, 6, 32, 90, 12, 2, 59] (nine values)

This is the array from the past paper, question 3. With nine values the snapshots are long, so only the table after each pass is shown. Try predicting each row before you read it.



PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[50, 83, 76, 6, 32, 90, 12, 2, 59]	—	—
1	$j = 0..7$	[50, 76, 6, 32, 83, 12, 2, 59, 90]	8	6
2	$j = 0..6$	[50, 6, 32, 76, 12, 2, 59, 83, 90]	7	5
3	$j = 0..5$	[6, 32, 50, 12, 2, 59, 76, 83, 90]	6	5
4	$j = 0..4$	[6, 32, 12, 2, 50, 59, 76, 83, 90]	5	2
5	$j = 0..3$	[6, 12, 2, 32, 50, 59, 76, 83, 90]	4	2
6	$j = 0..2$	[6, 2, 12, 32, 50, 59, 76, 83, 90]	3	1

PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
7	j = 0..1	[2, 6, 12, 32, 50, 59, 76, 83, 90]	2	1
8	j = 0 only	[2, 6, 12, 32, 50, 59, 76, 83, 90] (no swaps: stop)	1	0
Total		sorted	36	22

36 comparisons and 22 swaps (= the 22 inversions). Count how many passes it needed: the 2 sits at index 7 and must walk to index 0, one place per pass.

Common mistakes

Sorting / Bubble sort · Common mistakes

Common mistakes

- Letting the inner loop run to the end every time.**
With `j < arr.length - 1` the program still works but re-checks the locked tail. Use `j < arr.length - 1 - pass` so each pass is shorter.
- Reading past the end.**
The comparison uses `arr[j + 1]`, so `j` must stop at `length - 2`. A loop condition of `j < arr.length` throws `ArrayIndexOutOfBoundsException`.
- Swapping without a temporary variable.**
`a[j] = a[j + 1]; a[j + 1] = a[j];` leaves two equal values and loses one. Always save one value in `temp` first.
- Using $\geq$ instead of $>$.**
Equal values then swap with each other. That wastes swaps and destroys stability.
- Forgetting to reset swapped at the start of each pass.**
If it stays true from an earlier pass, the early exit never triggers and the sort loses its best-case speed.

- 6 Writing the array after every swap when asked for “after each pass”.**
A pass is the whole walk. Record one array per pass, and count swaps and comparisons separately.
-
- 7 Reversing the loops for descending order.**
Only the comparison changes ($<$ instead of $>$). The loops, the range and the flag stay the same.
-
- 8 Stopping early for the wrong reason.**
Stop when a *whole pass* makes no swaps, not when one pair happens to be in order.
-

A test-ready summary

Sorting / Bubble sort · A test-ready summary

A test-ready summary

TEST ANSWER

Bubble sort is a stable, in-place, comparison-based sorting algorithm. It repeatedly walks through the array, compares each pair of neighbouring values and swaps them when they are in the wrong order, so after each pass the largest remaining value has moved to its final place at the right. Each pass is one cell shorter than the one before, and with a **swapped** flag it stops as soon as a pass makes no swaps. The number of swaps equals the number of inversions. Its best-case time is $O(n)$ for sorted input, and its average and worst cases are $O(n^2)$ with $O(1)$ extra space. It is suitable for teaching and tiny arrays, but not for large or random data.

Practice

Sorting / Bubble sort · Practice

Practice

Guided question

Use bubble sort (ascending, with the early-exit flag) on [6, 3, 8, 4]. For each pass write every comparison, then the array at the end of the pass.

▼ Reveal the guided answer

Pass 1: 6 > 3 swap; 6 > 8 keep; 8 > 4 swap → [3, 6, 4, 8]

Pass 2: 3 > 6 keep; 6 > 4 swap → [3, 4, 6, 8]

Pass 3: 3 > 4 keep → [3, 4, 6, 8]

Totals: 6 comparisons and 3 swaps, in 3 passes. The last pass made no swaps, so the algorithm stopped.

Independent question

Trace bubble sort on [9, 4, 7, 3, 5].

- ☐ Predict the number of swaps first, using the inversions.
- ☐ Write the array after every pass.
- ☐ Record the comparisons and swaps in each pass.
- ☐ Say why the algorithm stopped.

▼ Reveal the independent answer

Inversions: 7, so exactly 7 swaps are needed.

PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[9, 4, 7, 3, 5]	—	—
1	j = 0..3	[4, 7, 3, 5, 9]	4	4
2	j = 0..2	[4, 3, 5, 7, 9]	3	2
3	j = 0..1	[3, 4, 5, 7, 9]	2	1

PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
4	j = 0 only	[3, 4, 5, 7, 9] (<i>no swaps: stop</i>)	1	0
Total		sorted	10	7

Test-style question

Show the contents of the array [50, 83, 76, 6, 32, 90, 12, 2, 59] at the end of each pass when sorting **descending** with bubble sort. (This is the past paper, question 3(b), with bubble sort instead of insertion sort.)

▼ Reveal the test-style answer

PASS	PAIRS COMPARED	ARRAY AFTER THE PASS	COMPARISONS	SWAPS
Start	—	[50, 83, 76, 6, 32, 90, 12, 2, 59]	—	—
1	j = 0..7	[83, 76, 50, 32, 90, 12, 6, 59, 2]	8	6
2	j = 0..6	[83, 76, 50, 90, 32, 12, 59, 6, 2]	7	2
3	j = 0..5	[83, 76, 90, 50, 32, 59, 12, 6, 2]	6	2
4	j = 0..4	[83, 90, 76, 50, 59, 32, 12, 6, 2]	5	2
5	j = 0..3	[90, 83, 76, 59, 50, 32, 12, 6, 2]	4	2
6	j = 0..2	[90, 83, 76, 59, 50, 32, 12, 6, 2] (<i>no swaps: stop</i>)	3	0
Total		sorted	33	14

Remember: descending means the *smallest* value of the active range is carried to the right end on each pass.

More worked cases: edge cases, duplicates and descending order

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted

1

2

4

3

5

01234

↓ ascending bubble sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[1,2,4,3,5]	1 > 2 → false; leave the pair; pass 1; 2 > 4 → false; leave the pair; pass 1; 4 > 3 → true; swap adjacent values; pass 1; 4 > 5 → false; leave the pair; pass 1	[1,2,3,4,5]	4
2	[1,2,3,4,5]	1 > 2 → false; leave the pair; pass 2; 2 > 3 → false; leave the pair; pass 2; 3 > 4 → false; leave the pair; pass 2	[1,2,3,4,5]	3

7 key comparisons; 1 swaps.

2. Reverse order

5

4

3

2

1

01234

↓ ascending bubble sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[5,4,3,2,1]	5 > 4 → true; swap adjacent values; pass 1; 5 > 3 → true; swap adjacent values; pass 1; 5 > 2 → true; swap adjacent values; pass 1; 5 > 1 → true; swap adjacent values; pass 1	[4,3,2,1,5]	4
2	[4,3,2,1,5]	4 > 3 → true; swap adjacent values; pass 2; 4 > 2 → true; swap adjacent values; pass 2; 4 > 1 → true; swap adjacent values; pass 2	[3,2,1,4,5]	3
3	[3,2,1,4,5]	3 > 2 → true; swap adjacent values; pass 3; 3 > 1 → true; swap adjacent values; pass 3	[2,1,3,4,5]	2
4	[2,1,3,4,5]	2 > 1 → true; swap adjacent values; pass 4	[1,2,3,4,5]	1

10 key comparisons; 10 swaps.

3. Duplicate values

2

2

1

3

0123

↓ ascending bubble sort

1

2

2

3

0123

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[2,2,1,3]	2 > 2 → false; leave the pair; pass 1; 2 > 1 → true; swap adjacent values; pass 1; 2 > 3 → false; leave the pair; pass 1	[2,1,2,3]	3
2	[2,1,2,3]	2 > 1 → true; swap adjacent values; pass 2; 2 > 2 → false; leave the pair; pass 2	[1,2,2,3]	2
3	[1,2,2,3]	1 > 2 → false; leave the pair; pass 3	[1,2,2,3]	1

6 key comparisons; 2 swaps.

4. Uneven length

38

27

43

3

9

0

1

2

3

4

↓ ascending bubble sort

3

9

27

38

43

0

1

2

3

4

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[38,27,43,3,9]	38 > 27 → true; swap adjacent values; pass 1; 38 > 43 → false; leave the pair; pass 1; 43 > 3 → true; swap adjacent values; pass 1; 43 > 9 → true; swap adjacent values; pass 1	[27,38,3,9,43]	4
2	[27,38,3,9,43]	27 > 38 → false; leave the pair; pass 2; 38 > 3 → true; swap adjacent values; pass 2; 38 > 9 → true; swap adjacent values; pass 2	[27,3,9,38,43]	3
3	[27,3,9,38,43]	27 > 3 → true; swap adjacent values; pass 3; 27 > 9 → true; swap adjacent values; pass 3	[3,9,27,38,43]	2
4	[3,9,27,38,43]	3 > 9 → false; leave the pair; pass 4	[3,9,27,38,43]	1

10 key comparisons; 7 swaps.

5. Descending target

8

3

5

1

0123

↓ descending bubble sort

8

5

3

1

0123

The trace below connects the input diagram to the final result.

PASS	BEFORE	EVALUATED CONDITIONS	AFTER	KEY COMPARISONS
1	[8,3,5,1]	8 < 3 → false; leave the pair; pass 1; 3 < 5 → true; swap adjacent values; pass 1; 3 < 1 → false; leave the pair; pass 1	[8,5,3,1]	3
2	[8,5,3,1]	8 < 5 → false; leave the pair; pass 2; 5 < 3 → false; leave the pair; pass 2	[8,5,3,1]	2

5 key comparisons; 1 swaps. Change `arr[j] > arr[j + 1]` to `arr[j] < arr[j + 1]`. Each pass then locks the smallest active value on the right.

Exam-style practice with worked answers

▼ A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

PASS	BEFORE	AFTER	KEY COMPARISONS
1	[8,3,5,1]	[8,5,3,1]	3
2	[8,5,3,1]	[8,5,3,1]	2

Final result [8,5,3,1]. Change $\text{arr}[j] > \text{arr}[j + 1]$ to $\text{arr}[j] < \text{arr}[j + 1]$. Each pass then locks the smallest active value on the right.

▼ B. Explain stability with tagged equal keys. [4]

Strict inversion comparisons do not move a value past an equal one. Using $>=$ (or $<=$ descending) would remove that guarantee.

▼ C. Give the time and space, then justify both. [5]

Best: $O(n)$. Already sorted input: one pass makes $n - 1$ comparisons and no swaps. Average / worst: $O(n^2)$. A reverse-order array needs many shrinking sweeps. Auxiliary space: $O(1)$ iterative; $O(n)$ recursive worst case. A temporary swap value versus a chain of calls. Stable / in place: Yes / yes. Strict $>$ avoids crossing equals; values are exchanged inside the input. Adaptive: Yes in this optimized version. The swapped flag enables early stop.

▼ D. Debug the code before tracing it. [4]

MISTAKE	SYMPTOM	FIX
Loop to $j < \text{arr.length}$	$\text{arr}[j + 1]$ reaches an invalid index.	Use $j < \text{arr.length} - 1$ - pass.
Declare swapped only once	An earlier swap prevents later early termination.	Reset it to false before every pass.

▼ E. What happens for [] and [7]? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.